

SDE.

Build full-stack & mobile software where **evals confirm the code follows the intent** — not just that it runs. With autonomous handoff and a path to a self-running software factory.

How it works

Example 1 · SaaS

Example 2 · Payment gateway

AI now writes working code in seconds. The open question is no longer “does it run?” — it’s “did it build what we meant?”

— SDE moves the unit of trust from a passing test to a trustworthy eval.

“Tests pass” is not “intent satisfied”

A unit test checks an **example**. It can stay green while the code quietly betrays the **intent** — because it never exercised the case that matters.

- Pagination that **duplicates a row** under a concurrent insert
- A query that **leaks another tenant’s data**
- A charge endpoint that **bills twice** on a retry

	CORRECT IMPL	THE MUTANT
unit test	PASS	PASS
the eval	PASS	FAIL

The mutant is a **plausible wrong implementation**. The unit test can’t tell it apart from the right one. A **validity-aware eval** can.

Three pillars

PILLAR 1

Eval validity is the product

An eval is trusted only when it is GREEN-on-right $\wedge$ RED-on-wrong — it passes on the intended code and is proven to FAIL on a plausible mutant.

PILLAR 2

Progressive rigor

One framework, three tiers: Tier 0 convention $\rightarrow$ Tier 1 gated skills $\rightarrow$ Tier 2 autonomous factory. You never install machinery you don't need yet.

PILLAR 3

Proof by dogfooding

It is built using itself. The proof is runnable; resumability is demonstrated by a cold agent recovering state from disk alone.

The lifecycle: objective → eval-confirmed code

OBJECTIVE → decomposed into thin vertical **slices**, each run through:

spec → bind evals → trust → freeze → execute → eval-run → done

Every step is a **pure function of disk state** — a `grep` / `diff` / `cat`, not a value held in the chat. The chat is a cache; the repo is the memory. A fresh session computes the same “what’s next?”.

A spec is numbered intent statements

```
.sde/slices/01-pagination/spec.md  
  
# what we MEAN — not how. each intent binds ≥1 eval.  
I1. Every item in a paging session is returned  
    exactly once — no omission, no duplicate.  
    eval: journey evals/session.py gate=required  
         mutant=evals/mutant.diff  
I2. Paging is stable under a concurrent insert  
    (keyset, not OFFSET).  
    eval: journey evals/session.py gate=required  
    eval: ux      evals/rubric.md   gate=flag
```

WHY PROSE + BINDINGS

Intent, then evidence

Each intent is a single checkable statement. Each binds an eval that must catch a violation of it. Coverage is derived: every intent → ≥1 eval, or the slice can't advance.

Gates: **required** blocks · **flag** surfaces · advisory always allows.

An eval earns trust: **GREEN-on-right** $\wedge$ **RED-on-wrong**

```
evals/session.py + evals/mutant.diff  
  
# the eval (validity-aware)  
page 1 → concurrent insert → page rest  
assert every item appears exactly once  
  
# the mutant (authored eval-blind)  
- keyset cursor on (created_at, id)  
+ ORDER BY created_at LIMIT / OFFSET
```

A vacuous eval (`assert True`) is green-on-right — but green-on-wrong too. So it is **rejected**.

	CORRECT IMPL	THE MUTANT
unit test	PASS	PASS
the eval	PASS	FAIL

KILL RECORD

RED-on-mutant is the proof

The mutant the eval catches IS its evidence of specificity.
Recorded in trust.log; invalidated when the eval or intent changes
(drift → re-verify).

The information barrier

THE MUTANT AUTHOR SEES

- The **intent** (plain prose)
- The **code under test**

...and must write a plausible diff that violates the intent.

THE MUTANT AUTHOR NEVER SEES

- The **eval** it must beat

Blind to the assertion, it can't craft a **strawman** tuned to be caught. In Tier 2 this is an **eval-blind git worktree**.

Gold standard: a **surgical** mutant that breaks one intent facet while every **other** eval stays green — proof the eval is load-bearing and non-redundant.

The bounded intent-audit

An independent auditor sees **only the intent + the eval set** — never the code — and must **exhibit a betrayer**: an implementation that passes every eval yet violates a named facet.

It answers the question trust alone can't: *is anything intended but un-evaluated?*

- **Betrayer found** → required new eval
- **2 clean rounds** → satisfied
- **3-round cap** → escalate, never spin

Every fact is **PRIMITIVE** or **DERIVED**

PRIMITIVE — AUTHORED ONCE

Written, never recomputed

objective · spec · spec.lock · tasks · trust.log · runs.log · decisions.
Append-only where it counts, so a torn write is the worst damage.

DERIVED — PURE F(DISK)

Computed at read time, never stored

phase · coverage · eval-trust · the next action · the handoff dossier.
Nothing to corrupt, nothing to go stale, no dual source of truth.

This is what makes “the next action is a pure function of disk” **literally true** — and why a fresh agent always agrees with the last one.

The ladder — resume is derivation

0	spec	≥1 intent line
1	bind	every intent has an eval
2	trust	killed $\wedge$ passed @ current hash
3	freeze	spec.lock == spec (diff)
4	execute	ready tasks done
5	eval-run	required verdicts PASS
6	done	no drift

The **first failing rung** is the phase; its action is the next action.

The resume protocol and the deterministic deriver walk the **same** ladder — so they can never diverge.

Hand off before rot — proven, not promised

2

cold agents recovered objective + next action + eval status from **disk alone**

0

chat history needed — the repo is the memory, the session is a cache

rung 2

trust evaluated **by hand** from RESUME.md; matched the deriver exactly

No token-fraction guesswork. Every step ends at a **committed, clean boundary**, so handing off “too early” costs only one cheap re-derivation. A stale handoff is detected by its git fingerprint and ignored.

One framework, three tiers

TIER 0

Discipline

Plain files; your own runner. The filesystem layout **is** the state machine. Minutes to adopt, no runtime.



TIER 1

Gated

Eval-trust + coverage + the ladder, enforced by 8 slash-command skills and a tested read-only deriver.



TIER 2

Factory

Bounded autonomous loop — fresh context per rung, caps, escalation, kill-switch. The software factory.



Operator's-eye view: who does what

THE HUMAN DECIDES

- The **objective** and how to slice it
- Ambiguous intent (auth, money, ordering)
- “Done when” — the factory escalates, never self-declares
- When to arm the kill-switch / steer

THE AGENT / FACTORY DOES

- Draft specs, write evals, author eval-blind mutants
- Implement slices; run the suite; write verdicts
- Derive the next action; hand off cleanly
- Stop at every committed boundary

Eight slash commands, one ladder

/SDE-INIT

scaffold

Create .sde/, capture the objective.

/SDE-SPEC

intents

Author the slice's intent statements.

/SDE-EVAL

trust ★

Bind evals + the eval-blind mutant barrier.

/SDE-NEXT

drive

Do the one derived action, then checkpoint.

/SDE-VERIFY

gate

Run the suite + the intent-audit.

/SDE-HANDOFF

freeze

Stamp the dossier for a clean resume.

/SDE-RESUME

cold start

Recover state from disk, continue.

/SDE-STATUS

read

Derived phase, coverage, trust, next.

Each is a thin layer over the on-disk state + the driver, so they can't drift from each other. `/sde-factory` wraps `/sde-next` for Tier 2.

From zero to a gated slice

```
Tier 0 · pure convention  
  
# Tier 0 — no install  
cp -r framework/templates/.sde ./sde  
# edit objective.md + slices/01-*/spec.md  
# write evals + a mutant diff per intent  
# run them with YOUR runner; follow RESUME.md
```

```
Tier 1 · gated  
  
# Tier 1 — Claude Code skills  
/sde-init "page the list, every item once"  
/sde-spec → /sde-eval → /sde-next  
/sde-verify  
/sde-resume      # after any context reset
```

Any step may run in a fresh session. `/sde-resume` continues from the derived next action.

The repository is the memory

```
project/.sde
.sde/
├─ objective.md      # north star + “done when”
├─ decisions.md      # append-only ADRs
├─ RESUME.md         # the ladder (run by hand or sde)
├─ runs.log          # append-only verdicts
└─ slices/01-.../
    ├─ spec.md spec.lock.md
    ├─ tasks.md trust.log
    └─ evals/
```

GREPPABLE, CATABLE

No black box

Read the whole state with cat. The deriver only reads it; it stores nothing.

App code lives in your normal tree (`src/`, `evals/`). **.sde/** holds state, not code.

A mechanical loop, fresh context per rung

```
framework/bin/sde-factory.sh

while sde next; do
  rc=$?                                # 0 done · 2 work · 1 error
  [ $rc = 0 ] && escalate done-when    # never auto-succeed
  before=$(git rev-parse HEAD)
  claude -p "do ONLY the next action; commit; stop"
  [ HEAD = before ] && escalate no-progress
done
```

EXECUTOR IN THE HARNESS

Agents propose; the driver decides

Only the driver runs evals/mutants and writes the logs — a builder can't grade its own work.

NO SESSION ROTS

One rung = one fresh claude -p

Nothing lives long enough to decay; the loop is the durable part, not the session.

Bounded autonomy — designed to halt

HARD STOPS

Caps everywhere

HEAD-based no-progress retry cap + a single MAX_STEPS budget + a .sde/STOP kill-switch + a steer file.

NEVER WEAKEN A GATE

Escalate, don't cheat

A gate that won't pass after N tries blocks + escalates. The loop never lowers a bar to get green.

STRUCTURAL GUARDS

Cheating can't pay

Weakening an eval re-opens its trust; forged verdicts can't (driver writes them); a vacuous eval can never reach done.

The driver **commits to a branch and never pushes or deploys**. Humans own “done.”

01

Example 1 — Multi-tenant SaaS

Web + API

Illustrative walkthrough

A team project tracker. The intent that ordinary tests miss: a user must never see another organization's data.

One objective, four slices

OBJECTIVE

Ship a multi-tenant tracker

Organizations sign up, invite teammates, and manage projects — each org's data fully isolated from every other.

- 01 Auth & org membership
- 02 Projects list — tenant isolation ★
- 03 Invite teammates — roles
- 04 Billing — subscription

The intent the tests forget

```
.sde/slices/02-projects/spec.md
```

- I1. A user only ever sees projects that belong to **their own organization**.
eval: contract evals/isolation.py
gate=**required** mutant=evals/leak.diff
- I2. Members see all org projects; the org boundary is never crossed by a filter, a search, or a direct id lookup.

WHY IT'S DANGEROUS

Silent, not loud

A tenant leak doesn't crash. It returns data — the wrong org's. Nothing in a single-tenant test suite notices.

The intent is a **negative** ("never sees..."). Negatives are exactly what example-based tests under-cover.

The eval seeds two tenants

```
src/projects.sql + evals/isolation.py

# the mutant: forgets the tenant filter
- WHERE owner_id=? AND org_id=?
+ WHERE owner_id=?                # leaks

# the eval: seed org A + org B
login as a member of org A
GET /projects → assert 0 rows from org B
```

The unit test seeds **one** org, so both impls return the same rows — green.

	WITH FILTER	WITHOUT
unit test · 1 org	PASS	PASS
isolation eval · 2 orgs	PASS	FAIL

Illustrative. The mutant (a dropped `AND org_id=?`) is the textbook multi-tenant leak.

Trust → freeze → execute

✓	bind	isolation.py bound to I1
✓	trust	leak.diff killed it · barrier on
✓	freeze	spec locked
→	execute	implement the scoped query

ORDER MATTERS

Eval before code

The eval is trusted — and red — before the query is written. The first job of the executor is to turn it green the right way.

Each rung ends in a commit. If context fills here, a fresh `/sde-resume` picks up at **execute**.

The leak is caught before it ships

Implementation lands. `/sde-verify` runs the suite: the isolation eval goes **green**; the intent-audit finds no betrayer. Slice 02 derives **done**.

A regression that drops the filter later → eval red → the gate blocks the merge. The intent is now permanently guarded.

WHAT THE EVAL BOUGHT

A **negative intent** ("never another org's data") turned into an executable, regression-proof guard — the one the unit suite structurally couldn't express.

02

Example 2 — Payment gateway

Backend

Merchant mobile app

Admin dashboard

Illustrative walkthrough

One objective spanning three surfaces. Each surface has an intent that is cheap to get wrong and expensive to ship wrong.

Three surfaces, one objective

Backend

Idempotent charges · money rounding · webhooks — the ledger of record

Mobile

Merchant checkout · never show “Paid” before the gateway confirms — Maestro journey

Admin

Payouts & reconciliation · only finance-role may move money — RBAC

Slices are **vertical**: each spans the surface end-to-end and carries its own trusted evals. The factory runs them with disjoint write sets.

The same payment, submitted twice

INTENT I1

Charge exactly once

A retry, a flaky network, a double-tap — submitting the same payment twice must bill the customer once, and return the same charge.

This is the single most expensive bug class in payments — and the easiest to pass a happy-path test.

The contract is the **idempotency key**: same key in, same charge out, no second row.

The eval replays the key

```
src/charges.py
POST /charges   Idempotency-Key: abc

# mutant – ignores the key
charge = create(amount)      # → 2 rows
# right – atomic upsert on the key
charge = get_or_create(key, amount)
```

The unit test sends **one** request. The intent is about the **second**.

	CORRECT IMPL	THE MUTANT
unit · 1 request	PASS	PASS
eval · 2× same key	PASS	FAIL

WHAT THE EVAL ASSERTS

One charge, same id

Two requests, identical key → exactly one row, and the same charge id returned both times.

Never show “Paid” before the gateway confirms

```
evals/checkout.maestro.yaml  
  
appId: com.paybridge.merchant  
- launchApp  
- tapOn: "Charge $20"  
# mutant: optimistic UI marks Paid on tap  
- assertVisible: "Declined" # test card  
- assertNotVisible: "Paid"
```

Documented contract, not a device run. The runner-agnostic journey (*flow* → {*verdict*, *artifacts*}) has an executable API-level analogue.

	CONFIRM-FIRST	OPTIMISTIC
unit · render receipt	PASS	PASS
journey · decline	PASS	FAIL

THE INTENT

Truth over optimism

A declined card must read “Declined.” Showing “Paid” before settlement is the violation the journey eval pins.

Only finance-role can move money

```
src/payouts.py + evals/authz.py  
  
# mutant: missing the role check  
POST /payouts → create_payout() # any admin  
  
# right: guard the endpoint  
require_role("finance")  
  
# the eval: an authz matrix  
as ops → expect 403  
as finance → expect 200
```

CORRECT IMPL

THE MUTANT

unit · finance pays

PASS

PASS

authz · ops blocked

PASS

FAIL

Illustrative. A missing role check passes every "happy path" test — finance can still pay out. Only the **negative** case (ops must be blocked) catches it.

One loop drives all three surfaces

• backend · mobile · admin

Disjoint slice directories → the factory advances them with no write contention. Each rung is a fresh, information-scoped `claude -p`; mutants are authored in an eval-blind worktree; a different-model judge grades the “clear error message” semantic eval at rungs 2 and 5.

WHEN A GATE WON'T PASS

The slice is **blocked + escalated** to the human queue — the loop moves to an independent slice or stops. It never weakens the idempotency eval to go green.

Three surfaces, three intent violations caught

BACKEND

Double charge

An idempotency-blind handler billed twice on a retry — green on the 1-request test, red on the replay eval.

MOBILE

Optimistic "Paid"

The app showed success before settlement — green on the receipt test, red on the decline journey.

ADMIN

Missing authz

A payout endpoint any admin could call — green on the finance happy-path, red on the ops-blocked matrix.

Each is a **negative or second-order** intent — the exact shape ordinary tests under-cover, and exactly what a trusted eval pins.

What is proven vs. designed

PROVEN · RUNS IN THE REPO

- Eval-trust on real code (pagination): mutant green on tests, red on eval
- The deriver is tested (6/6); resumability shown by two cold agents
- The framework dogfoods itself — state resumes from disk

DESIGNED · NOT YET RUN HERE

- The Tier-2 factory's unattended run (driver written, not executed)
- A real mobile device journey; a live different-model judge
- **The two examples are illustrative** walkthroughs of the method

Read it, run the proof, slice your objective

```
start here  
  
# see an eval catch an intent violation  
bash examples/01-pagination/run.sh  
  
# watch the framework derive its own state  
python3 framework/bin/sde status --root .
```

[FRAMEWORK.md](#)

[examples/01-pagination](#)

[docs/reasoning/00-04](#)

Code generation was never the hard part.
Knowing it built what you meant is — and the eval is what knows.